1. Title

TrustShare : Secure File-Sharing System

2. Objective

Build a secure file-sharing platform that allows users to upload, store, and share files safely through encrypted access mechanisms.

The system supports secure file uploads, server-side AES-256 encrypted file storage, controlled sharing permissions, temporary access links, download tracking, audit logging, and authentication-based access control.

The platform is designed to improve digital file security by protecting sensitive documents and enabling secure collaboration between users.

This solution can be used for organizations, educational institutions, businesses, secure document exchange, and enterprise collaboration systems.

Outcomes

(i) Developed and deployed a secure file-sharing and document management platform.

(ii) Implemented authentication and role-based access control systems.

(iii) Built secure file upload and encrypted storage workflows.

(iv) Developed secure sharing and temporary access management systems.

(v) Implemented file activity monitoring and audit logging workflows.

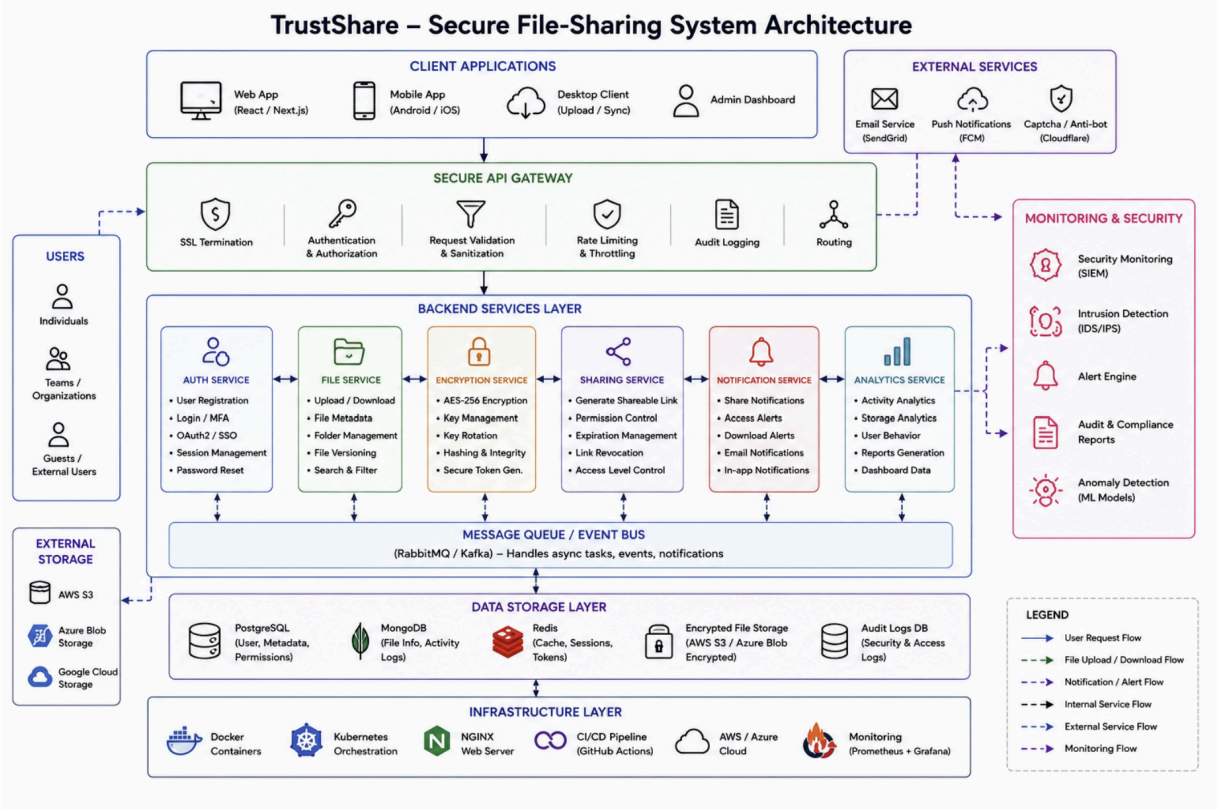
(vi) Built notification and security alert systems.

(vii) Developed analytics dashboards for storage usage and activity monitoring.

(viii) Implemented server-side AES-256 encryption and secure key management workflows.

(ix) Deployed the platform using Docker and cloud deployment platforms such as AWS or Azure.

3. Architecture Components



4. Modules to be Implemented

1. User Authentication Module

(i) User registration and login

(ii) Multi-factor authentication

(iii) Password recovery

(iv) Session management

(v) JWT authentication

(vi) OAuth2 integration

2. File Management Module

(i) Secure file upload

(ii) File categorization

(iii) Folder management

(iv) File search and filtering

(v) File version management

(vi) File metadata management

3. Secure Sharing Module

(i) Shareable secure links

(ii) Permission management

(iii) Temporary access expiration

(iv) Access-level configuration

(v) Download restrictions

(vi) Share activity monitoring

4. Encryption & Security Module

The platform follows a Server-Side Encryption Architecture.

Files are uploaded from the React/Next.js frontend to the FastAPI backend through HTTPS.

Before any file is stored in cloud storage, FastAPI encrypts the file using AES-256 encryption.

Only encrypted files are stored in cloud storage.

Encryption Workflow

(i) User uploads file through the web application.

(ii) FastAPI validates file type, size, and permissions.

(iii) FastAPI generates a unique AES-256 encryption key.

(iv) The uploaded file is encrypted on the server.

(v) The encrypted file is stored in AWS S3 or Azure Blob Storage.

(vi) File metadata and permissions are stored in PostgreSQL.

(vii) Activity logs are stored in MongoDB.

(viii) Authorized users request access using secure sharing links.

(ix) FastAPI decrypts the file temporarily in memory.

(x) The decrypted file is securely delivered to the authorized user.

Security Features

(i) AES-256 Encryption

(ii) HTTPS/TLS Communication

(iii) JWT Authentication

(iv) OAuth2 Authentication

(v) Role-Based Access Control

(vi) Temporary Share Links

(vii) Download Tracking

(viii) Audit Logging

(ix) Secure Token Generation

(x) Key Rotation

Key Management

(i) Unique encryption key per uploaded file.

(ii) Keys managed securely on the FastAPI server.

(iii) Keys are never exposed to users.

(iv) Periodic key rotation improves security.

5. Access Monitoring Module

(i) Download tracking

(ii) File access history

(iii) Login activity monitoring

(iv) Audit logs

(v) Security event monitoring

(vi) Suspicious activity detection

6. Notification Module

(i) File-sharing notifications

(ii) Access alerts

(iii) Security warnings

(iv) Email notifications

(v) Download notifications

(vi) Expiration reminders

7. Analytics Dashboard Module

File Analytics

(i) Storage usage statistics

(ii) File upload reports

(iii) Download analytics

(iv) Sharing activity reports

Security Dashboard

(i) Login monitoring

(ii) Unauthorized access attempts

(iii) Security event reports

(iv) Audit monitoring

Admin Dashboard

(i) User activity monitoring

(ii) Storage utilization

(iii) Security analytics

(iv) Sharing reports

(v) System monitoring

5. Week-wise Module Implementation and High-Level Requirements

Milestone 1: Week 1 & 2 — Project Initialization, Design Process & Core Setup

Tasks

(i) Define project objectives and secure collaboration workflows.

(ii) Design secure system architecture and database schema.

(iii) Create UI wireframes and workflow planning.

(iv) Setup React/Next.js frontend and FastAPI backend.

(v) Configure PostgreSQL, MongoDB, and Redis.

(vi) Implement authentication and role-based access system.

(vii) Build secure file upload workflows.

(viii) Develop file management dashboard.

Outcomes

(i) Understand secure file-sharing and collaboration workflows.

(ii) Learn secure architecture and database design concepts.

(iii) Build frontend and backend project initialization.

(iv) Working authentication and file management system.

Milestone 2: Week 3 & 4 — Encryption & Secure Sharing

Tasks

(i) Implement AES-256 server-side encryption workflows.

(ii) Integrate secure cloud storage.

(iii) Develop encrypted file retrieval workflows.

(iv) Build secure sharing system.

(v) Implement access permissions and access control.

(vi) Add temporary link expiration workflows.

(vii) Implement encryption key management.

(viii) Build secure file decryption workflows.

Outcomes

(i) Implement secure encryption and storage systems.

(ii) Build secure file-sharing and access management workflows.

(iii) Understand secure communication and collaboration concepts.

(iv) Generate protected file-sharing functionalities.

(v) Implement secure key management workflows.

Milestone 3: Week 5 & 6 — Monitoring, Notifications & Analytics

Tasks

(i) Develop activity monitoring system.

(ii) Implement audit logging workflows.

(iii) Build notification and security alert system.

(iv) Generate file activity and security reports.

(v) Develop storage analytics and usage monitoring dashboard.

(vi) Implement suspicious activity detection.

Outcomes

(i) Build security monitoring and alert systems.

(ii) Implement analytics and activity reporting workflows.

(iii) Understand secure monitoring and audit management concepts.

(iv) Complete end-to-end secure file-sharing workflows.

Milestone 4: Week 7 & 8 — Testing, Deployment & Documentation

Tasks

(i) Perform security testing and workflow validation.

(ii) Improve UI responsiveness and system optimization.

(iii) Deploy platform using Docker and cloud environments.

(iv) Configure production infrastructure.

(v) Prepare final project documentation and presentation.

(vi) Demonstrate the complete TrustShare platform.

Outcomes

(i) Gain deployment and security testing experience.

(ii) Improve platform stability and usability.

(iii) Complete live deployment and final demonstration.

(iv) Prepare professional project documentation and presentation.

6. Evaluation Criteria

Milestone 1 (Week 2)

(i) Project initialization and architecture setup completed.

(ii) Authentication and file management implemented.

(iii) Secure upload workflows functional.

(iv) System design and UI planning completed.

Milestone 2 (Week 4)

(i) Encryption and secure sharing workflows implemented.

(ii) Access control and permission management functional.

(iii) Temporary link expiration workflows working.

(iv) Secure storage integration completed.

Milestone 3 (Week 6)

(i) Monitoring and notification system implemented.

(ii) Audit logging and activity tracking functional.

(iii) Analytics dashboard and reports generated.

(iv) Security monitoring workflows integrated.

Milestone 4 (Week 8)

(i) Fully deployed frontend and backend.

(ii) Security testing and validation completed.

(iii) Documentation and presentation prepared.

(iv) Successful end-to-end platform demonstration completed.

7. Tools & Tech Stack

Programming Language

Backend

(i) Python

(ii) FastAPI

Frontend

(i) React.js / Next.js

Database

(i) PostgreSQL

(ii) MongoDB

(iii) Redis

Security & Encryption

(i) JWT Authentication

(ii) OAuth2 Authentication

(iii) AES-256 Encryption

(iv) bcrypt Password Hashing

(v) HTTPS/TLS Security

(vi) Secure Token Generation

Cloud & Storage

(i) AWS S3

(ii) Azure Blob Storage

(iii) Docker

(iv) AWS / Azure

Libraries & Frameworks

Backend

(i) FastAPI

(ii) SQLAlchemy

(iii) Pydantic

(iv) Alembic

(v) Uvicorn

(vi) Cryptography Library

(vii) Redis

Frontend

(i) React.js

(ii) Next.js

(iii) Tailwind CSS

(iv) Axios

(v) JWT Authentication

Dev & Deployment Tools

(i) VS Code

(ii) Git + GitHub

(iii) Docker & Docker Compose

(iv) AWS / Azure

(v) Postman

(vi) Monitoring & Logging Tools

8. Performance Metrics

Security Performance

(i) Encryption and decryption speed

(ii) Unauthorized access detection accuracy

(iii) Secure sharing reliability

(iv) Key management efficiency

File Management Performance

(i) File upload speed

(ii) Download response time

(iii) Storage optimization efficiency

(iv) File retrieval efficiency

System Performance

(i) API response time

(ii) Concurrent file-sharing handling

(iii) Database query optimization

(iv) Secure file processing speed

9. Example Quantitative Goals

Secure File Sharing

Achieve secure and reliable encrypted file-sharing workflows.

Access Monitoring

Detect unauthorized access attempts with high accuracy.

File Management

Maintain fast file upload and download performance with secure storage.

Platform Performance

Support multiple concurrent users with stable and secure collaboration workflows.